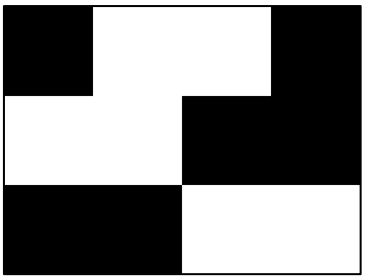
CONNECTEDNESS

Example

Binary image:

0 1 1 0
1 1 0 0

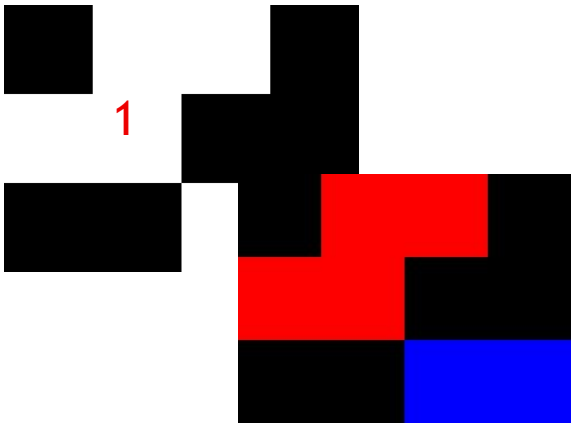
0 0 1 1



For 4 connectivity method, there are two objects are found.

After labeling:

0 1 1 0
1 1 0 0
0 0 2 2



- Object 1 → Label 1
- Object 2 → Label 2
- **Count** = 2 objects

Exercise 1: Using 8-connectivity, label all connected components in the same binary image, count the total number of objects present, and display the labeled matrix as output.

Exercise 2: A satellite captures a binary image of a farmland where 1 represents regions of crop growth and 0 represents barren land. Using 4-connectivity, agricultural analysts need to identify distinct crop clusters. For the image:

0	1	1	0	1	0
0	1	1	0	1	0
0	0	1	1	0	0
1	1	0	0	1	1

Identify and tell how many crop clusters are there?

Exercise 3

A drone captures a binary image (11×11) of a forest region where 1 represents trees and 0 represents empty ground. Using **4-connectivity**, forestry analysts need to identify distinct tree clusters.

0 0 1 1 0 0 0 1 0 0 0
0 1 1 1 0 0 0 1 0 1 0

0 1 0 0 0 1 1 1 0 1 0
0 0 0 0 0 0 0 0 0 1 0
1 1 0 0 1 1 0 0 0 0 0
1 1 0 0 0 1 0 0 1 1 0
0 0 0 0 0 0 0 0 1 1 0
0 0 0 1 1 1 0 0 0 0 0
0 0 0 1 0 1 0 0 0 0 0

0 0 0 1 1 1 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0

SIZE FILTERING

Exercise 1 :

Binary Image (1 = object pixel, 0 = background):
0 1 1 0 0
0 1 1 0 1
0 0 0 0 1
1 0 0 0 0

1 0 1 1 1

Step 1 – Label Objects (8-connectivity):

0 1 1 0 0

0 1 1 0 2

0 0 0 0 2
3 0 0 0 0

3 0 4 4 4

- Object 1 → 4 pixels
- Object 2 → 3 pixels
- Object 3 → 2 pixels
- Object 4 → 3 pixels

Step 2 – Apply Size Filtering:

Threshold = Keep objects ≥ **3 pixels**
Remove Object 3 (size = 2 pixels)

Output Image After Size Filtering:

0 1 1 0 0
0 1 1 0 2
0 0 0 0 2

0 0 0 0 0
0 0 4 4 4

CHAIN CODE

Example

Consider a simple binary image of a square

object: 0 0 0 0 0

0 1 1 1 0
0 1 1 1 0

0 1 1 1 0
0 0 0 0 0

- The boundary pixels are the ones with 1 touching the background.
- If we start from the top-left corner, we trace clockwise → right, down, left, up.
- Chain code might be: 000111222333 (right, down, left,

up). This sequence completely describes the square.

Boundary pixels (outer ring of 1s):

(1,1),(1,2),(1,3),(2,3),(3,3),(3,2),(3,1),(2,1)

Freeman Code

3	2	1
4	*	0
5	6	7

NW	N	NE
W	*	E
SW	S	SE

Initialization

- **Start pixel P_0**: (1,1) (topmost-leftmost 1).
- **Backtrack** (pixel we "came from" before starting): pretend we came from the **west** → (1,0).
  (This just sets the starting search direction.)

Trace (clockwise)

I'll show: **Step k — Current → Next | Direction | Chain code**
(We only list the first found boundary neighbor each time; that's the move we take.)

1. **P_0 (1,1)** → (1,2) | E | **0**
2. **P_1 (1,2)** → (1,3) | E | **0**
3. **P_2 (1,3)** → (2,3) | S | **6**
4. **P_3 (2,3)** → (3,3) | S | **6**
5. **P_4 (3,3)** → (3,2) | W | **4**
6. **P_5 (3,2)** → (3,1) | W | **4**
7. **P_6 (3,1)** → (2,1) | N | **2**
8. **P_7 (2,1)** → (1,1) | N | **2** (back to start → **stop**)

We've returned to the start pixel with the same traversal orientation, so the loop is closed.

Final Freeman Code Output

0 , 0, 6, 6, 4, 4, 2, 2

Boundary point sequence

(1,1) → (1,2) → (1,3) → (2,3) → (3,3) → (3,2) → (3,1) → (2,1) → back to (1,1)

MOMENT

The simplest type of moment is the **raw moment**, which is defined by the following equation for a discrete image:

$$M_{pq} = \sum_x \sum_y x^p y^q I(x, y)$$

Here, p and q are the moment orders. By varying p and q,
you can calculate different moments that reveal different information about the image:

- Zeroth-order moment (M00):
- This is the sum of all pixel intensities. In a binary image, where pixels are either 0 or 1, M00 gives the **area** of the object.

2. **First-order moments:**
   - Along x-axis:

$$M_{10} = \sum_x \sum_y x \cdot I(x, y)$$

   - Along y-axis:

$$M_{01} = \sum_x \sum_y y \cdot I(x, y)$$

These are used to compute the **centroid (center of mass)** of the shape:

$$\bar{x} = \frac{M_{10}}{M_{00}}, \quad \bar{y} = \frac{M_{01}}{M_{00}}$$

3. **Second-order moments:**

$$M_{20} = \sum_x \sum_y x^2 \cdot I(x, y)$$

- $$M_{02} = \sum_x \sum_y y^2 \cdot I(x, y)$$
- $$M_{11} = \sum_x \sum_y xy \cdot I(x, y)$$

These describe the **spread and orientation** of the object around the centroid.

Applications

Raw Moment

Zeroth order = Area

$$M_{pq} = \sum_x \sum_y x^p y^q I(x, y)$$

Example Binary Image (3×3)

$$I(x, y) = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

Object pixels (value = 1):

$$(1, 2),\ (2, 1),\ (2, 2),\ (3, 2)$$

Each has $I(x, y) = 1$.

$$M_{00} = 1 + 1 + 1 + 1 = 4$$

$$M_{00} = 4$$

First order = Center

Raw Moment

$$M_{pq} = \sum_x \sum_y x^p y^q I(x,y)$$

$$I(x,y) = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

$(1,2),\ (2,1),\ (2,2),\ (3,2)$

First-order moments:

- Along x-axis:

$$M_{10} = \sum_x \sum_y x \cdot I(x,y)$$

$$M_{10} = (1\cdot 1) + (2\cdot 1) + (2\cdot 1) + (3\cdot 1)$$

$$M_{10} = 1 + 2 + 2 + 3 = 8$$

- Along y-axis:

$$M_{01} = \sum_x \sum_y y \cdot I(x,y)$$

$$M_{01} = (2\cdot 1) + (1\cdot 1) + (2\cdot 1) + (2\cdot 1)$$

$$M_{01} = 2 + 1 + 2 + 2 = 7$$

$$\bar{x} = \frac{M_{10}}{M_{00}}, \quad \bar{y} = \frac{M_{01}}{M_{00}}$$

$$M_{10} = 8, \quad M_{01} = 7$$

$$\bar{x} = \frac{M_{10}}{M_{00}} = \frac{8}{4} = 2$$

$$\bar{y} = \frac{M_{01}}{M_{00}} = \frac{7}{4} = 1.75$$

center of the object = $(2, 1.75)$.

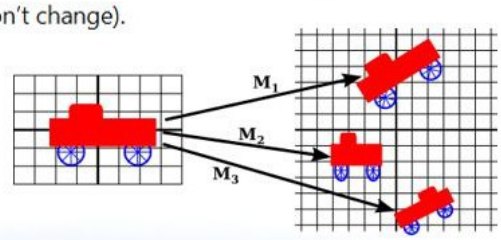
Central Moments

$$\mu_{pq} = \sum_x \sum_y (x - x_c)^p (y - y_c)^q I(x,y)$$

where:

- $x_c = \frac{M_{10}}{M_{00}}$, $y_c = \frac{M_{01}}{M_{00}}$ are the centroid coordinates.
- p, q are orders.

They are called **central moments** because they are computed **relative to the centroid**, making them **translation invariant** (object shifted in the image → values don't change).



1. μ_{20}

Contributions:

- (1,2): $(1-2)^2 = 1$
- (2,1): $(2-2)^2 = 0$
- (2,2): $(2-2)^2 = 0$
- (3,2): $(3-2)^2 = 1$

$$\mu_{20} = \sum (x - x_c)^2 I(x,y)$$

$$\mu_{20} = 1 + 0 + 0 + 1 = 2$$

2. μ_{02}

Contributions:

- (1,2): $(2-1.75)^2 = 0.0625$
- (2,1): $(1-1.75)^2 = 0.5625$
- (2,2): $(2-1.75)^2 = 0.0625$
- (3,2): $(2-1.75)^2 = 0.0625$

$$\mu_{02} = \sum (y - y_c)^2 I(x,y)$$

$$\mu_{02} = 0.0625 + 0.5625 + 0.0625 + 0.0625 = 0.75$$

3. μ_{11}

Contributions:

- (1,2): $(1-2)(2-1.75) = (-1)(0.25) = -0.25$
- (2,1): $(0)(-0.75) = 0$
- (2,2): $(0)(0.25) = 0$
- (3,2): $(1)(0.25) = 0.25$

$$\mu_{11} = \sum (x - x_c)(y - y_c) I(x,y)$$

$$\mu_{11} = -0.25 + 0 + 0 + 0.25 = 0$$

HOUGH TRANSFORM

1. **Slope–Intercept Form:**

   $y = mx + c$

   Here, m is the slope and c is the y-intercept of the line.

2. **Point–Slope Form:**

   $y - y_1 = m(x - x_1)$

   Used when the slope m and a point (x_1, y_1) on the line are known.

3. **Two–Point Form:**

   $\frac{y-y_1}{y_2-y_1} = \frac{x-x_1}{x_2-x_1}$

   Represents a line passing through two known points (x_1, y_1) and (x_2, y_2).

4. **Normal (Polar) Form:**

   $\rho = x\cos\theta + y\sin\theta$

   Commonly used in the **Hough Transform**, where ρ is the perpendicular distance from the origin to the line, and θ is the angle of the perpendicular.

5. **General Form:**

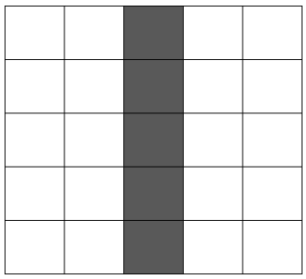
   $Ax + By + C = 0$

   A more general equation where A, B, and C are constants representing the line's orientation and position.

Example 1 :

5×5 Binary Image (Row, Column Form)

0 0 1 0 0
0 0 1 0 0
0 0 1 0 0
0 0 1 0 0
0 0 1 0 0

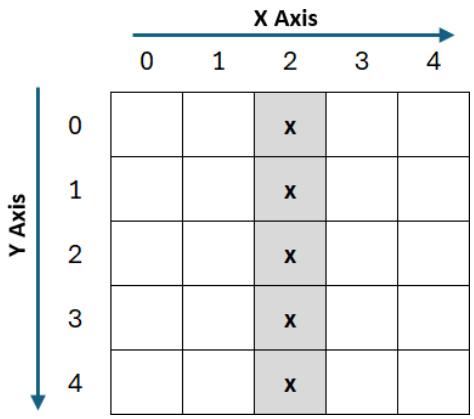


Step 1 :Coordinates of edge pixels (row, column):

$$(0,2), (1,2), (2,2), (3,2), (4,2)$$

This clearly shows a vertical line at column 2.

For (x,y) co-ordinate,



Coordinates of edge pixels (x,y):

(2,0),(2,1),(2,2),(2,3),(2,4)

Hough Transform Equation

$$\rho = x\cos\ \theta + y\sin\ \theta$$

We'll check three simple angles: θ = 0°, 45°, 90°

$\cos\ 0^\circ = 1,\qquad \sin\ 0^\circ = 0$

$\cos\ 45^\circ = 0.7071,\quad \sin\ 45^\circ = 0.7071$

$\cos\ 90^\circ = 0,\qquad \sin\ 90^\circ = 1$

Step 2: Compute ρ for each pixel

Hough Transform ρ Computation Table

Pixel (x,y)	ρ @ θ = 0°	ρ @ θ = 45°	ρ @ θ = 90°
(2, 0)	$2 \times 1 + 0 \times 0 = 2$	$2 \times 0.7071 + 0 \times 0.7071 = 1.4142$	$2 \times 0 + 0 \times 1 = 0$
(2, 1)	$2 \times 1 + 1 \times 0 = 2$	$2 \times 0.7071 + 1 \times 0.7071 = 2.1213$	$0 + 1 \times 1 = 1$
(2, 2)	$2 \times 1 + 2 \times 0 = 2$	$2 \times 0.7071 + 2 \times 0.7071 = 2.8284$	$0 + 2 \times 1 = 2$
(2, 3)	$2 \times 1 + 3 \times 0 = 2$	$2 \times 0.7071 + 3 \times 0.7071 = 3.5355$	$0 + 3 \times 1 = 3$
(2, 4)	$2 \times 1 + 4 \times 0 = 2$	$2 \times 0.7071 + 4 \times 0.7071 = 4.2426$	$0 + 4 \times 1 = 4$

Each cell counts how many pixels voted for that (ρ, θ).

Pixel (x,y)	ρ (0°)	ρ (45°)	ρ (90°)
(2,0)	2.0000	1.4142	0
(2,1)	2.0000	2.1213	1
(2,2)	2.0000	2.8284	2
(2,3)	2.0000	3.5355	3
(2,4)	2.0000	4.2426	4

After rounding ρ to the nearest integer:

Pixel (x,y)	ρ (0°)	ρ (45°)	ρ (90°)
(2,0)	2	1	0
(2,1)	2	2	1
(2,2)	2	3	2
(2,3)	2	4	3
(2,4)	2	4	4

Step 3: Accumulator Table (Voting result)

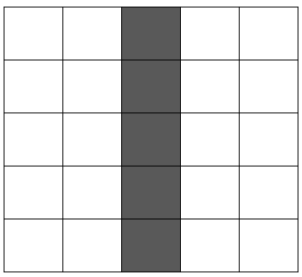
ρ ↓ / θ →	0°	45°	90°
0	0	0	1
1	0	1	1
2	5	1	1
3	0	1	1
4	0	2	1

Step 4: Interpretation

- The **strong peak** is at **(ρ = 2, θ = 0°) → 5 Votes,**
- That means "There is a vertical line located at column 2."
- All other cells have fewer votes, so they represent weaker or no lines.

Conclusion

- Each edge pixel "votes" for possible line parameters (ρ, θ).
- Votes are stored in a 2-D **accumulator table**.
- The cell with maximum votes marks the detected line.



Example 2

Suppose the noise can be present in the image.

0 0 1 0 0

0 0 1 0 1

0 0 1 0 0

0 0 1 0 1

0 0 1 0 0

Coordinates of Edge pixels:

(2,0), (2,1), (4,1), (2,2), (2,3), (4,3), (2,4) — 7 points.

Pixel (x,y)	ρ@0°	ρ@45° (rounded)	ρ@90°
(2,0)	2	1	0
(2,1)	2	2	1
(4,1)	4	4	1
(2,2)	2	3	2
(2,3)	2	4	3
(4,3)	4	5	3
(2,4)	2	4	4

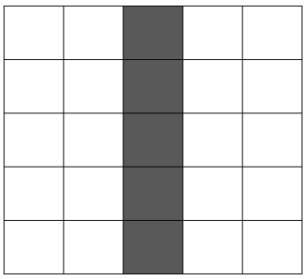
Accumulator Table (Voting result)

ρ ↓ / θ →	0°	45°	90°
0	0	0	1
1	0	1	2
2	5	1	1
3	0	1	2
4	2	3	1
5	0	1	0

Detection result (peak analysis)

- The **strong peak** is at **(ρ = 2, θ = 0°) → 5 Votes,**

Final Output :



Example 3

0 0 0 0 1

0 0 0 0 0

1 1 1 1 1

0 0 0 0 0

1 0 0 0 0

So the edge points are:

$$(4,0),\ (0,2),\ (1,2),\ (2,2),\ (3,2),\ (4,2),\ (0,4)$$

I compute ρ for $\theta = 0^\circ,\ 45^\circ,\ 90^\circ$, then round ρ to the nearest integer (quantization) to build votes.

2) Rounded ρ (nearest integer)

Pixel (x,y)	ρ@0°	ρ@45° (rounded)	ρ@90°
(4,0)	4	3	0
(0,2)	0	1	2
(1,2)	1	2	2
(2,2)	2	3	2
(3,2)	3	4	2
(4,2)	4	4	2
(0,4)	0	3	4

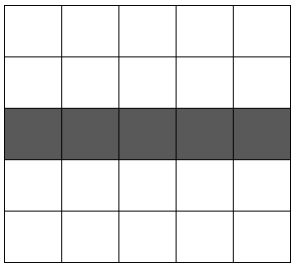
4) 2-D Accumulator (rows = θ, cols = ρ=0..4)

ρ ↓ / θ →	0°	45°	90°
0	2	0	1
1	1	1	0
2	1	1	5
3	1	3	0
4	2	2	1

Detection / Conclusion

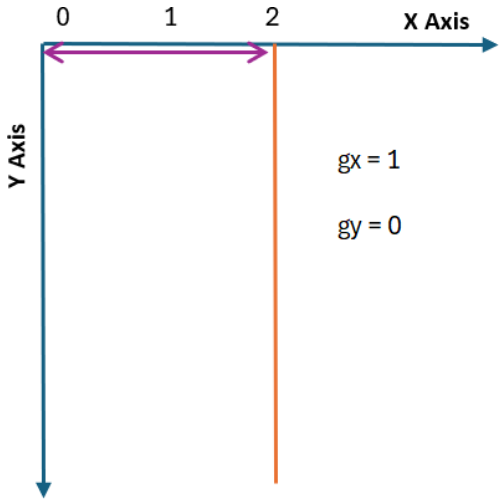
- The strongest accumulator peak is at **(ρ = 2, θ = 90°)** with **5 votes** → this corresponds to the **horizontal line** $y = 2$.

Final Output

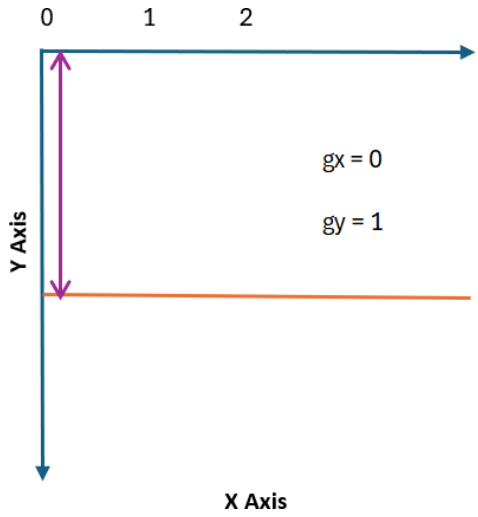


FOOT-OF-NORMAL METHOD

For vertical line



For Horizontal line



🧠 Step 3. Geometric meaning

Line type	Edge direction	Gradient (normal) direction	Example (g_x, g_y)
Vertical (x = const)	Vertical	Horizontal	(1, 0)
Horizontal (y = const)	Horizontal	Vertical	(0, 1)
Diagonal y = x	45° line	Perpendicular (135°)	(1, -1)
Diagonal y = -x	-45° line	Perpendicular (45°)	(1, 1)

Mathematical way to find gx and gy.

$$\boxed{ax + by + c = 0}$$

Line equation is $x = 2$

$x - 2 = 0$

$(1)x + (0)y - 2 = 0$

X coefficient is dx , ie. 1

Y coefficient is dy ie. 0

Example Problem :

0 0 1 0 0

0 0 1 0 0

0 0 1 0 0

0 0 1 0 0

0 0 1 0 0

Edge pixels (x,y) using 0-based coordinates (column, row): (2,0), (2,1), (2,2), (2,3), (2,4)

---

1. Geometric line found directly

For a **general straight line**, we can use:

$$y = mx + b$$

where

- m= slope
- b= intercept

But for vertical line, slope becomes **infinite** (because Δx = 0).

To check:

Pick any two points:

$$(2,0)\text{and}(2,1)$$

$$m = \frac{y_2 - y_1}{x_2 - x_1} = \frac{1 - 0}{2 - 2} = \frac{1}{0} \rightarrow \text{undefined } (\infty)$$

So it's a vertical line. The equation is:

$$x = 2$$

Look at all the **x-values**:

$$x = [2,2,2,2,2]$$

and all the **y-values**:

$$y = [0,1,2,3,4]$$

Notice:

All x-values are same, $x = 2$

y-values are changing

All points share the same x coordinate → they lie on the vertical line

$$\boxed{x = 2}$$

(or in standard form $1 \cdot x + 0 \cdot y - 2 = 0$).

---

2. Foot-of-normal parameterization (the method)

Idea: For any line $ax + by + c = 0$, the foot of the perpendicular from the origin $(0,0)$onto that line is

$$(x_0, y_0) = \left(\frac{-ac}{a^2 + b^2}, \frac{-bc}{a^2 + b^2}\right).$$

All edge pixels lying on the same line will vote at the same foot point (x_0, y_0)in the parameter/image-space; a strong accumulator peak there means the line exists.

Apply to our line $x - 2 = 0$(so $a = 1$, $b = 0$, $c = -2$):

$$x_0 = \frac{-1 \cdot (-2)}{1^2 + 0^2} = 2, y_0 = \frac{-0 \cdot (-2)}{1} = 0.$$

$$\boxed{(x_0, y_0) = (2,\ 0).}$$

So every edge pixel votes at (2,0)

Or

1. Find gradients (g_x,g_y)
2. For each edge pixel (x,y), Find Find v, x_0 and y_0.

$$\boxed{v = \frac{x\, g_x + y\, g_y}{g_x^2 + g_y^2}} \qquad \boxed{x_0 = v\, g_x, y_0 = v\, g_y.}$$

So foot point is (x_0, y_0)

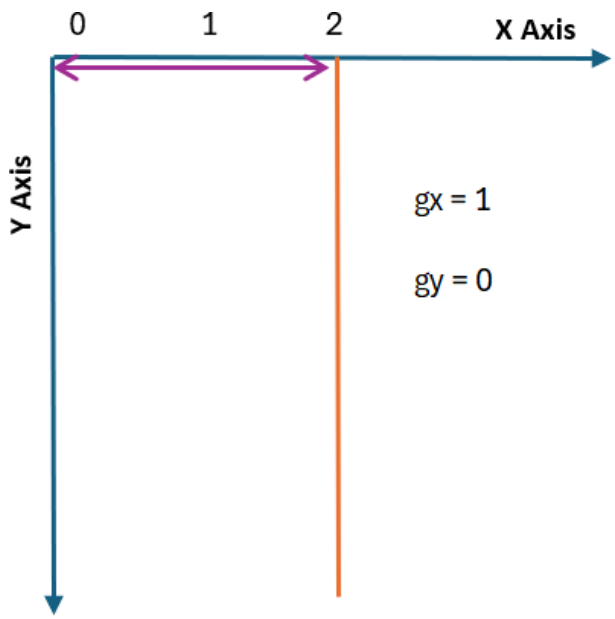
For each edge pixel $(x, y)$with gradient components (g_x, g_y)compute

$$v = \frac{x\, g_x + y\, g_y}{g_x^2 + g_y^2}$$

then the **foot of the normal** (the voting position in parameter space) is

$$x_0 = v\, g_x, y_0 = v\, g_y.$$

Each pixel contributes one vote at the parameter-pixel (x_0, y_0). Peaks in the parameter image (counts of (x_0, y_0)) mark candidate lines.



That single accumulator cell (2,0) receives all votes → **peak** → detection of the line $x = 2$.

---

2) Choose/estimate gradients (g_x, g_y)

We must supply a local gradient at each edge pixel. For this simple binary image the edge is a **vertical column** at column $x = 2$, so the gradient is horizontal (points along ±x). A standard simple choice is

$$(g_x, g_y) = (1, 0)$$

for all edge pixels (unit horizontal gradient). (In practice you compute these from Sobel/finite differences; using the unit vector is sufficient to show how votes behave.)

3) Compute votes — explicit table

We compute v, x_0, $y_0$for each pixel using $(g_x, g_y) = (1,0)$.

Pixel (x,y)	g_x, g_y	$v = (xg_x + yg_y)/(g_x^2 + g_y^2)$	$x_0 = vg_x$	$y_0 = vg_y$	Vote location (x_0, y_0)
(2,0)	(1,0)	$v = (2 \cdot 1 + 0 \cdot 0)/(1) = 2$	$x_0 = 2$	$y_0 = 0$	(2, 0)
(2,1)	(1,0)	$v = (2 \cdot 1 + 1 \cdot 0)/(1) = 2$	$x_0 = 2$	$y_0 = 0$	(2, 0)
(2,2)	(1,0)	$v = (2 \cdot 1 + 2 \cdot 0)/(1) = 2$	$x_0 = 2$	$y_0 = 0$	(2, 0)
(2,3)	(1,0)	$v = (2 \cdot 1 + 3 \cdot 0)/(1) = 2$	$x_0 = 2$	$y_0 = 0$	(2, 0)
(2,4)	(1,0)	$v = (2 \cdot 1 + 4 \cdot 0)/(1) = 2$	$x_0 = 2$	$y_0 = 0$	(2, 0)

So **every pixel votes at the same parameter point** $(x_0, y_0) = (2,0)$.

---

4) Accumulator (parameter image) and peak

Count votes at each (x_0, y_0)bin:

- Bin (2,0) : **5 votes**
- All other bins : 0

Thus the parameter-space peak is at $(x_0, y_0) = (2,0)$.

---

5) Convert foot (x_0, y_0)back to line equation

The foot $(x_0, y_0)$is the point on the line nearest the origin. We can convert it to the usual normal form

$$x\cos\,\theta + y\sin\,\theta = \rho,$$

where $\rho = \sqrt{x_0^2 + y_0^2}$and $\theta = \text{atan2}\,(y_0, x_0)$.

For $(x_0, y_0) = (2,0)$:

- $\rho = \sqrt{2^2 + 0^2} = 2$
- $\theta = \text{atan2}\,(0,2) = 0$

So the line equation is

$$x\cos\ 0 + y\sin\ 0 = 2 \Longrightarrow x = 2.$$

This matches the obvious column at $x = 2$.

---

6) Remarks / robustness notes

- In real images you compute (g_x, g_y)per pixel (e.g., Sobel). If the gradient sign were negative (e.g., $(g_x, g_y) = (-1,0)$), all votes would still fall at $(x_0, y_0) = (-2,0)$(the foot on the opposite side). You then convert that foot to $(\rho, \theta)$and still recover the same geometric line $x = 2$.
- You should quantize (bin) the continuous (x_0, y_0)coordinates to build a discrete accumulator array; adjacent votes within one bin add up to a peak.
- For slanted lines the foot coordinates will differ for different pixels, but they cluster at one $(x_0, y_0)$and produce a peak. After finding the peak you can refine orientation by using the contributing pixels (the book suggests making them form a θ histogram for better angular accuracy).

Example 2

Find: foot of perpendicular from $P(3,2)$to the line

$$L: 2x + y - 8 = 0$$

Let's compute:

Given:

$$A = 2, B = 1, C = -8, x_1 = 3, y_1 = 2$$
$$A^2 + B^2 = 2^2 + 1^2 = 5$$

---

x_0:

$$x_0 = \frac{B(Bx_1 - Ay_1) - AC}{A^2 + B^2}$$

$$= \frac{1(1 \times 3 - 2 \times 2) - 2(-8)}{5} = \frac{(3 - 4) + 16}{5} = \frac{15}{5} = 3$$

y_0:

$$y_0 = \frac{A(-Bx_1 + Ay_1) - BC}{A^2 + B^2}$$

$$= \frac{2(-1 \times 3 + 2 \times 2) - 1(-8)}{5} = \frac{2(-3+4)+8}{5} = \frac{2(1)+8}{5} = \frac{10}{5} = 2$$

■ Foot of perpendicular:

$$(x_0, y_0) = (3,2)$$

That means the point (3,2) **already lies on the line** $2x + y - 8 = 0$.

Or you can use another method,

1. Find gradients (g_x,g_y)
2. For each edge pixel (x,y), Find Find v, x_0 and y_0.

$$\boxed{v = \frac{x\,g_x + y\,g_y}{g_x^2 + g_y^2}} \qquad \boxed{x_0 = v\,g_x, y_0 = v\,g_y.}$$

So foot point is (x_0, y_0)

Given $(x, y) = (3, 2), gx = 2, gy = 1$:

$$gx^2 + gy^2 = 2^2 + 1^2 = 4 + 1 = 5$$

$$v = \frac{3 \cdot 2 + 2 \cdot 1}{5} = \frac{6+2}{5} = \frac{8}{5}$$

$$x_0 = v \cdot gx = \frac{8}{5} \cdot 2 = \frac{16}{5}$$

$$y_0 = v \cdot gy = \frac{8}{5} \cdot 1 = \frac{8}{5}$$

The foot of the normal from $P(3, 2)$ to $2x + y - 8 = 0$ is:

$$\boxed{\left(\frac{16}{5}, \frac{8}{5}\right)}$$

Or numerically:

- $x_0 = 3.2$
- $y_0 = 1.6$

If your round the value into nearest integer you can get (3,2)

Foot of perpendicular:

$$(x_0, y_0) = (3,2)$$

Example

Suppose we have a **5×5 binary edge image** (1 = edge, 0 = no edge):

0 0 1 0 0

0 1 0 1 0

1 0 0 0 1

0 1 0 1 0

0 0 1 0 0

We want to detect **circles of radius r = 2**.

Step 2: Set up the accumulator

For CHT, the accumulator array stores votes for **possible circle centers** (a, b).

- Since our image is 5×5, possible centers are also 5×5.
- Initialize the accumulator A[a][b] = 0.

Use the circle equation

The circle equation is:

$$(x - a)^2 + (y - b)^2 = r^2$$

Where:

- (x, y) is an **edge pixel**,
- (a, b) is a **possible center**,
- r is the radius.

For **each edge pixel**, we find **all (a, b) that satisfy the circle equation** and increment the accumulator at those positions.

Step 3: Process one edge point

Take the **edge pixel at (0,2)** (row 0, column 2):

$$(x - a)^2 + (y - b)^2 = 2^2 = 4$$

Substitute x = 0, y = 2:

$$(0 - a)^2 + (2 - b)^2 = 4$$

Find the valid centers from possible centers

The possible centers are,

(0,0), (0,1), (0,2), (0,3), (0,4)

(1,0), (1,1), (1,2), (1,3), (1,4)

(2,0), (2,1), (2,2), (2,3), (2,4)

(3,0), (3,1), (3,2), (3,3), (3,4)

(4,0), (4,1), (4,2), (4,3), (4,4)

(a,b)	Calculation	Result
(0,0)	$(0-0)^2+(2-0)^2=4$	. Yes
(0,1)	$(0-0)^2+(2-1)^2=1$	+
(0,2)	$(0-0)^2+(2-2)^2=0$	+
(0,3)	$(0-0)^2+(2-3)^2=1$	+
(0,4)	$(0-0)^2+(2-4)^2=4$	. Yes
(1,2)	$(0-1)^2+(2-2)^2=1$	+
(2,2)	$(0-2)^2+(2-2)^2=4$	. Yes

■ So valid centers: **(0,0), (0,4), (2,2)**

It would be too long to write all 25, so the example shows some representative checks (like (0,0), (0,4), (2,2), (1,0), (2,4) etc.) , these are not satisfy the valid center.

In an **actual algorithm**, the computer **tests all 25** and increments votes for any that satisfy the circle equation.

We now check **all integer (a, b) positions** in 5×5 grid that satisfy this:

1. (a, b) = (0, 0) → (0-0)^2 + (2-0)^2 = 0 + 4 = 4 ▪

2. (a, b) = (0, 4) → (0-0)^2 + (2-4)^2 = 0 + 4 = 4 ■
3. (a, b) = (2, 2) → (0-2)^2 + (2-2)^2 = 4 + 0 = 4 ▪
4. (a, b) = (-2, 2) → invalid (outside image) +
5. (a, b) = (1, 0) → (0-1)^2 + (2-0)^2 = 1 + 4 = 5 +
6. (a, b) = (1,4) → (0-1)^2 + (2-4)^2 = 1 + 4 = 5 +
7. (a, b) = (2,0) → (0-2)^2 + (2-0)^2 = 4+4=8 +
8. (a, b) = (2,4) → (0-2)^2 + (2-4)^2 = 4+4=8 +

■ So valid centers for this edge pixel: **(0,0), (0,4), (2,2)**.

Increment accumulator at these positions.

Step 5: Process all edge pixels

Repeat for other edge pixels:

- (1,1), (1,3), (2,0), (2,4), (3,1), (3,3), (4,2)

After voting, the **accumulator might look like** (simplified counts):

A =

2	1	2	1	2
1	2	3	2	1
2	3	**4**	3	2
1	2	3	2	1
2	1	2	1	2

Step 6: Find circle centers

- Look for **local maxima** in accumulator.
- Maximum vote = 4 at **(2,2)**.

So the detected **circle center** is **(2,2)** with **radius = 2**.

Step 7: Result

- **Center:** (2,2)

- **Radius:** 2

This is exactly the circle formed by the edge pixels.

Center at (2,2) , each distance from 2 (ie radius)

0 0 **1** 0 0

0 **1** 0 **1** 0

1 0 **0** 0 **1**

0 **1** 0 **1** 0

0 0 **1** 0 0

RANSAC FOR STRAIGHT LINE DETECTION

Steps for RANSAC Line Fitting

1. **Input data points** (from binary or edge image).
2. **Set parameters:**
   - Number of iterations (N)
   - Distance threshold (t)
   - Minimum number of inliers required to accept a model (d)
3. **Repeat for N iterations:**
   1. Randomly **select two points** from the dataset.
   2. **Compute line parameters** (slope m, intercept c) passing through them.
   3. **Compute distance** of every other point to this line.
   4. **Count inliers** → points whose distance ≤ threshold t.
4. **Store the line model** if:
   - It has the **maximum number of inliers** so far.
5. **After all iterations**, select the **best model** (line with most inliers).
6. **Refit the line** using **only inlier points** (for improved accuracy).
7. **Output the final line equation** and optionally **highlight inliers/outliers**.

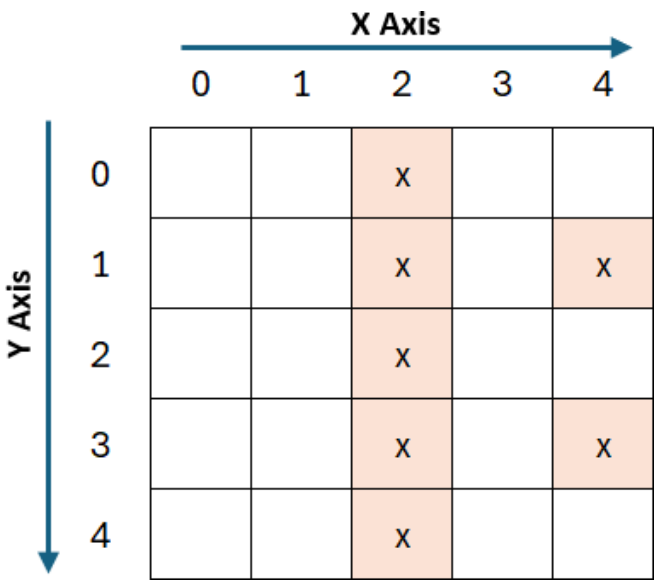
Steps

1. Pick **two random points** (minimum needed to define a line).
2. Draw the **line** passing through them.
3. That line is called a **candidate line** (a *hypothesis*).
4. Then, check how many other points lie close to it (called **inliers**).
5. Repeat for many random pairs — each new line is another **candidate line**.
6. Finally, choose the **best** candidate — the one with the **most inliers**.

```
0 0 1 0 0
0 0 1 0 1
0 0 1 0 0
0 0 1 0 1
0 0 1 0 0
```

Given edge pixels (row, column)

(0,2), (1,2), (1,4), (2,2), (3,2), (3,4), (4,2)



The x,y coordinate (2,0), (2,1), (4,1), (2,2), (2,3), (4,3), (2,4)

RANSAC parameters (chosen)

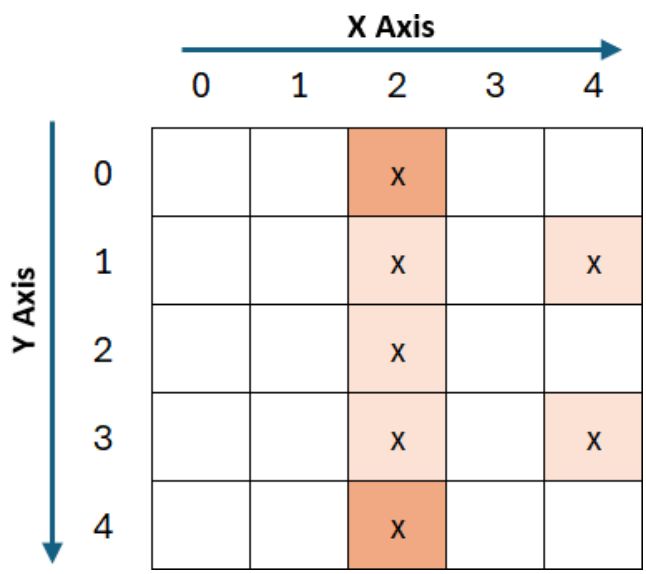
- Iterations $N = 10$(more than enough for this tiny set)

- Distance threshold $t = 0.5$pixels (a point is an inlier if its perpendicular distance to the candidate line ≤ 0.5)
- Minimum inliers to accept a line $d = 4$

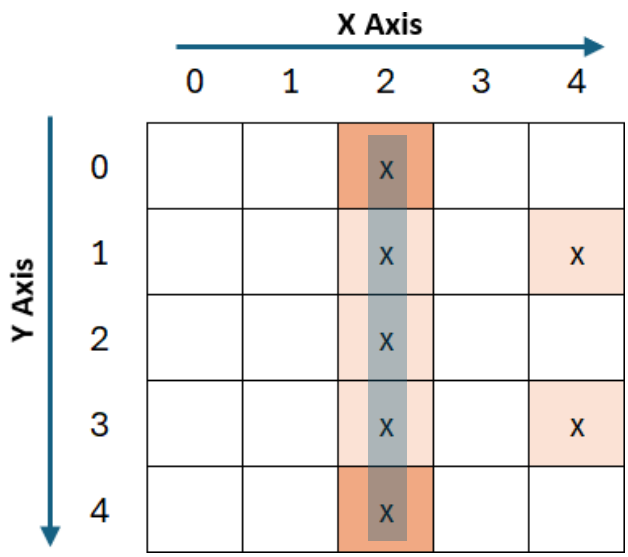
Step 1: Pick two random points (minimum needed to define a line).

✓ Pick two sample points: **(2,0)** and **(2,4)**.

✓ Those two define the vertical line at column = 2 (i.e., $x = 2$).



Step 2 : Draw the line passing through them.



Step 3 : That line is called a candidate line (a hypothesis).

- ✓ A candidate line simply means, a possible line that might fit the data.
- ✓ It's a hypothesis, not yet confirmed as the best one.

Step 4 : Find the line equation

Write the two points

$$(x_1, y_1) = (2,0)$$

$$(x_2, y_2) = (2,4)$$

Find slope m

$$m = \frac{y_2 - y_1}{x_2 - x_1} = \frac{4 - 0}{2 - 2} = \frac{4}{0}$$

But dividing by zero is not possible — this means the line is vertical.

In a vertical line,

- every point has the same x-value,
- but y can vary.

So, the line through both points has:

$$x = 2$$

That's the line equation.

Step 5 : Check how many other points lie close to it (called inliers).

Distances of every point to the candidate line $x = 2$

Distance to vertical line $x = 2$ is $|$ column(or x axis) $- 2$ $|$.

- (2,0): $|\ 2 - 2\ | = 0.0 \rightarrow$ **inlier**
- (2,1): $|\ 2 - 2\ | = 0.0 \rightarrow$ **inlier**
- (4,1): $|\ 4 - 2\ | = 2.0 \rightarrow$ **outlier**
- (2,2): $|\ 2 - 2\ | = 0.0 \rightarrow$ **inlier**
- (2,3): $|\ 2 - 2\ | = 0.0 \rightarrow$ **inlier**
- (4,3): $|\ 4 - 2\ | = 2.0 \rightarrow$ **outlier**
- (2,4): $|\ 2 - 2\ | = 0.0 \rightarrow$ **inlier**

Inlier count = 5 ($\geq$ d), so this model is excellent.

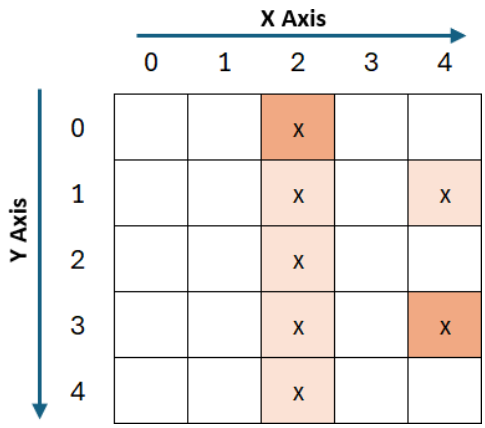
(Any other random pair that includes one of the noise points will yield a worse model with fewer inliers; so RANSAC will keep $x = 2$as best.)

For example,

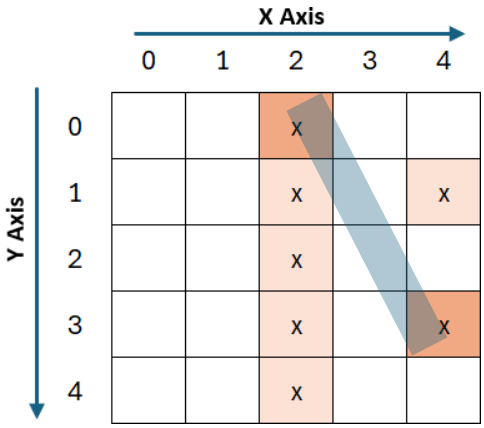
Next iteration

Pick another two random points.

Pick two sample points: **(0,2)** and **(3,4)**.



Draw candidate line



Find the line equation through (2,0) and (4,3)

We can use the **two-point form** of a line:

$$m = \frac{y_2 - y_1}{x_2 - x_1}$$

$$m = \frac{3-0}{4-2} = \frac{3}{2} = 1.5$$

Now find intercept c:

$$y = mx + c \Rightarrow 0 = 1.5(2) + c \Rightarrow c = -3$$

Line equation:

$$y = 1.5x - 3$$

◆ Step 2: Check all points (which are close or far)

We can compute the **predicted y** for each x, and see the **difference** (residual).

Point	Actual y	Predicted y = 1.5x - 3	Difference	Inlier? (tolerance ±0.5)
(2,0)	0	0	0	■ Inlier (seed point)
(2,1)	1	0	+1	+ Outlier
(4,1)	1	3	-2	+ Outlier
(2,2)	2	0	+2	+ Outlier
(2,3)	3	0	+3	+ Outlier
(4,3)	3	3	0	■ Inlier (seed point)
(2,4)	4	0	+4	+ Outlier

Step 3: Result

■ **Inliers:** (2,0), (4,3)

\+ **Outliers:** (2,1), (4,1), (2,2), (2,3), (2,4)

So this **candidate line** fits only the two points it was built from — **no other points lie close enough**.

Hence, **RANSAC will reject** this line and try another random pair next.

In this way RANSAC do the calculations for more iterations.

In our example, (2,0), (2,1), (4,1), (2,2), (2,3), (4,3), (2,4) : 7 values are there.

Total pairs = 7×6/2 = 21 possible pairs.

RANSAC might not test all 21 — maybe just 10 random tries —

RANSAC tries several pairs randomly:

Try #	Pair used	Candidate Line	Inliers Count	Decision
1	(2,0), (4,3)	y = 1.5x - 3	2	+ Reject
2	(2,1), (4,1)	y = 1	2	+ Reject
3	(2,0), (2,4)	x = 2	**5**	■ Best so far
4	(2,3), (4,1)	y = -x + 5	2	+ Reject
…	...	...	...	...
…	...	...	...	...
…	...	...	...	...

but if one pair like (2,0) & (2,4) gives a perfect line (x=2) with 5 inliers,

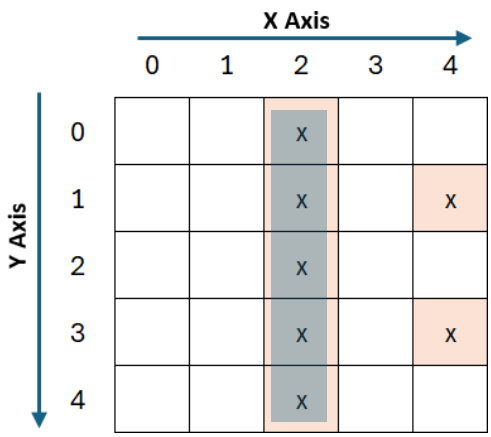
it will stop early or select that line as the best model.

Final RANSAC Output

Best line equation x=2

- **Reason:** inlier count = 5, distances 0; outliers distance = 2 (> threshold)

Inliers (2,0), (2,1), (2,2), (2,3), (2,4)




```
1 0 0 0 0 0 1
0 1 0 0 0 0 0
0 0 1 0 0 0 1
1 0 0 1 0 0 0
0 0 0 0 1 0 0
0 0 0 0 0 1 0
1 0 0 0 0 0 1
```

Points (from your 7×7 matrix):

(0,0), (6,0),

(1,1),

(2,2), (6,2),

(0,3), (3,3),

(4,4),

(5,5),

(0,6), (6,6)

We'll use the same RANSAC settings as before:

- minimum points to define a line = **2**
- distance threshold = **0.7** (pixels)
- distance from point to line computed by standard formula (see below)

Background formula (point-to-line distance)

For a line in form $ax + by + c = 0$, the perpendicular distance of point $(x, y)$is

$$d = \frac{| ax + by + c |}{\sqrt{a^2 + b^2}}.$$

A point is an **inlier** if $d \leq 0.7$.

Iteration A — good candidate

Random pair chosen: (0,0)and (6,6)

1) Line through the pair

Two equal increments in x and y ⇒ slope $m = (6 - 0)/(6 - 0) = 1$.

Equation: $y = x$.

Standard form: $x - y = 0 \rightarrow a = 1, b = -1, c = 0$.

Denominator $\sqrt{a^2 + b^2} = \sqrt{1^2 + (-1)^2} = \sqrt{2} \approx 1.41421356$.

2) Distances and inlier check

Compute $| x - y |/\sqrt{2}$for each point. Because threshold = 0.7,

equivalent inlier condition is $| x - y | \leq 0.7 \times \sqrt{2} \approx 0.9899$.

So only points with $| x - y | \leq 0.9899$(i.e. essentially $| x - y | = 0$) qualify.

Check each point quickly:

- (0,0): $| 0 - 0 | = 0 \Rightarrow d = 0 \leq 0.7 \rightarrow$ **inlier**
- (1,1): $| 1 - 1 | = 0 \rightarrow$ **inlier**
- (2,2): $| 2 - 2 | = 0 \rightarrow$ **inlier**
- (3,3): $| 3 - 3 | = 0 \rightarrow$ **inlier**
- (4,4): $| 4 - 4 | = 0 \rightarrow$ **inlier**
- (5,5): $| 5 - 5 | = 0 \rightarrow$ **inlier**
- (6,6): $| 6 - 6 | = 0 \rightarrow$ **inlier**

All others have $| x - y | \geq 1 \rightarrow$ not inliers.

Result (Iteration A):

- Inliers (7):

  (0,0), (1,1), (2,2), (3,3), (4,4), (5,5), (6,6)
- Outliers: (6,0), (6,2), (0,3), (0,6)

- **Inlier count = 7**

This is a **very good** candidate (matches the main diagonal $y = x$).

Iteration B — bad candidate

Random pair chosen: (6,0)and (0,3)

1) Line through the pair

Slope:

$$m = \frac{3-0}{0-6} = \frac{3}{-6} = -0.5.$$

Use point (6,0) to find intercept b:

$$0 = -0.5 \cdot 6 + b \Rightarrow b = 3.$$

So equation: $y = -0.5x + 3$.

Standard form: bring all terms to left:

$$0.5x + y - 3 = 0 (\text{so } a = 0.5, b = 1, c = -3).$$

Denominator $\sqrt{a^2 + b^2} = \sqrt{0.5^2 + 1^2} = \sqrt{0.25 + 1} = \sqrt{1.25} \approx 1.11803399$.

Distance threshold converted to numerator limit:

$$| ax + by + c | \leq 0.7 \times 1.11803399 \approx 0.7826.$$

2) Distances and inlier check (compute numerator $| 0.5x + y - 3 |$)

For each point:

- (0,0): numerator = $| 0 + 0 - 3 | = 3$. $d = 3/1.118 \approx 2.683 \rightarrow$ **outlier**
- (6,0): numerator = $| 3 + 0 - 3 | = 0$. $d = 0 \rightarrow$ **inlier** (seed)
- (1,1): numerator = $| 0.5 + 1 - 3 | = 1.5$. $d \approx 1.341 \rightarrow$ **outlier**
- (2,2): numerator = $| 1 + 2 - 3 | = 0$. $d = 0 \rightarrow$ **inlier**
- (6,2): numerator = $| 3 + 2 - 3 | = 2$. $d \approx 1.789 \rightarrow$ **outlier**
- (0,3): numerator = $| 0 + 3 - 3 | = 0$. $d = 0 \rightarrow$ **inlier** (seed)
- (3,3): numerator = $| 1.5 + 3 - 3 | = 1.5$. $d \approx 1.341 \rightarrow$ **outlier**
- (4,4): numerator = $| 2 + 4 - 3 | = 3$. $d \approx 2.683 \rightarrow$ **outlier**

- (5,5): numerator = $| 2.5 + 5 - 3 | = 4.5$. $d \approx 4.027 \rightarrow$ **outlier**
- (0,6): numerator = $| 0 + 6 - 3 | = 3$. $d \approx 2.683 \rightarrow$ **outlier**
- (6,6): numerator = $| 3 + 6 - 3 | = 6$. $d \approx 5.366 \rightarrow$ **outlier**

Result (Iteration B):

- Inliers (3): (6,0), (2,2), (0,3)
- Outliers: the other 8 points
- **Inlier count = 3**

This is a **poor** candidate compared with the diagonal.

Final decision after these two iterations

- Iteration A produced **inlier count = 7** (line $y = x$).
- Iteration B produced **inlier count = 3** (line $y = -0.5x + 3$).

RANSAC picks the candidate with the **maximum inliers**. **Therefore the good fit is the line from Iteration A:**

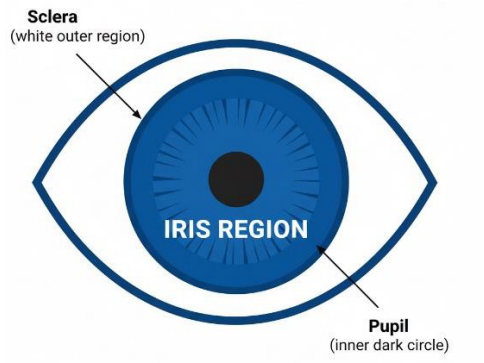
$$\boxed{y = x}$$

with inliers (0,0),(1,1),(2,2),(3,3),(4,4),(5,5),(6,6).

HUMAN IRIS LOCATION

1. Objective

✓ The goal is to automatically locate the iris region of a human eye in an image.



✓ Iris location is an important step in biometric identification systems, medical imaging, and computer vision applications like eye-tracking and emotion recognition.

2. Characteristics of the Iris

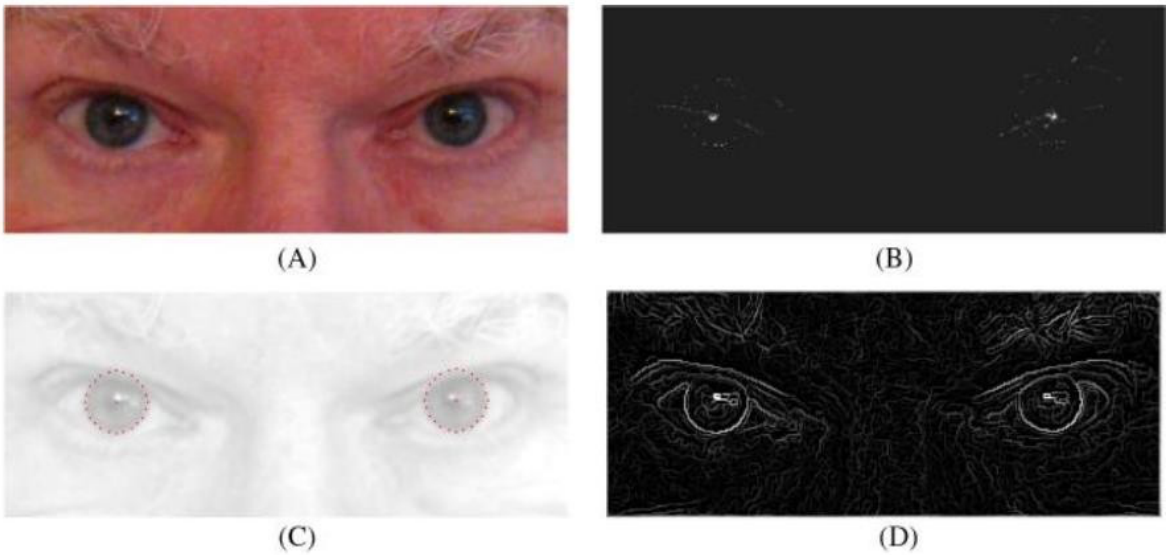
- The iris is approximately circular or elliptical in shape.
- It lies between:
  - Pupil (inner dark circle)
  - Sclera (white outer region)
- This distinct circular shape allows Hough Transform to be effectively applied for its detection.

Circle Hough Transform (CHT)

- Use Hough Transform for circles to detect:
  - Pupil boundary (inner circle)
  - Iris boundary (outer circle)
- The algorithm votes in (a, b, r) space, where:
  - (a, b) → center of the circle
  - r → radius of the circle

$$(x - a)^2 + (y - b)^2 = r^2$$

3. Steps for Iris Location Using Hough Transform



(A) Original Image

- This is a normal eye region photo.
- It shows both eyes clearly.
- The goal is to find the iris position in this image.

(B) Hough Transform Parameter Space

- Here, the image is processed using Hough Transform to find circular shapes.
- Each bright point represents a peak in parameter space → a possible iris center.
- This step tells the algorithm where the circle (iris) might be.

(C) Iris Detected

- Based on the peaks from (B), the algorithm draws a circular boundary around the iris.
- You can see dotted circles over both eyes → these show the accurately detected iris regions.

(D) Canny Edge Detection

- This image shows the edges found using Canny operator (a popular edge detector).
- It finds not only iris edges but many other edges (eyebrows, eyelids, wrinkles, shadows).

- These extra edges can interfere with detection if not handled properly.

Key Points

- Hough Transform finds circular shapes in the image.
- Gradient weighting (adding edge strength info) helps the algorithm focus more on strong iris edges and less on weak, unwanted edges.
- Even with other edges present, the algorithm successfully finds iris centers and draws accurate circles.

Why Hough Transform Works Well Here

- Iris boundary has **strong edge contrast**.
- Hough Transform is **robust to noise** and **partial occlusion** (e.g., eyelids, eyelashes).
- Voting mechanism helps locate the iris even when the circle is **not perfectly complete**.

Extensions

- If the iris is not a perfect circle (due to camera angle), Ellipse Hough Transform (EHT) or Generalized Hough Transform (GHT) can be applied.
- Speed improvement: Restrict the search range of radius to reduce computation.
- Post-processing may involve active contour models or segmentation to refine boundary.

Applications

- Biometric authentication (Iris recognition)
- Medical diagnostics (e.g., eye disease detection)
- Human–computer interaction (gaze tracking)
- Security systems

Summary:

- Iris detection is formulated as circular object detection.
- Hough Circle Transform locates the center and radius of the iris.
- This method is robust and forms the first step in iris recognition systems.